

Milestone 3: ARIA – Adaptive Runtime Index with Async-flush

Md Toki Tahmid COS568, Princeton University

Approach

The Milestone 2 naive hybrid flushes DPGM into LIPP synchronously, stalling every insert that triggers the threshold for the full duration of the migration. ARIA fixes this with two ideas: (1) the flush runs on a background thread so inserts are never blocked, and (2) the index decides at runtime whether to use the write buffer at all.

Runtime profiling. During the first 1024 operations after bulk load, ARIA counts inserts and lookups. If inserts exceed 50%, it stays in `WRITE_BUFFERED` mode; otherwise it drains the DPGM buffer into LIPP once and switches to `READ_OPTIMIZED` mode permanently. The profiling flag is cleared after this decision and never checked again.

WRITE_BUFFERED mode. Inserts go into `dpgm_active_`. When it reaches `flush_interval` keys (2M), ARIA atomically moves it to `dpgm_flushing_`, resets `dpgm_active_` to an empty buffer, and spawns a background thread to drain `dpgm_flushing_` into LIPP one key at a time. Incoming inserts continue into the fresh buffer without waiting. Lookups check LIPP, then `dpgm_active_`, then `dpgm_flushing_`. A `shared_mutex` protects LIPP during the flush: the background thread holds a `unique_lock`; concurrent lookups acquire a `shared_lock` only when the `flush_in_progress_` atomic flag (with acquire/release ordering) is set.

READ_OPTIMIZED mode. Both inserts and lookups go directly to LIPP. The hot path is a single predicted branch with no locking overhead, equivalent to running pure LIPP. The internal LIPP instance (MyLIPP) adds one `__builtin_expect` hint to the `find()` traversal loop, giving roughly 2.6% higher throughput over unmodified LIPP on the FB dataset.

No keys lost. Before switching to `READ_OPTIMIZED`, all keys in `dpgm_active_` are inserted into LIPP. During an async flush, keys in `dpgm_flushing_` remain reachable via the lookup path until the thread finishes. No key is ever discarded.

Hyperparameters. `pgm_error` was swept over {64, 128, 256} with `flush_interval` fixed at 2M. The variant with the highest average throughput across 3 runs was selected per workload.

Results

Figures ??-?? show throughput (Mops/s) and index size (GB) for both workloads across all three datasets. ARIA (labeled *HybridPGMLIPPAsync*) outperforms DPGM, LIPP, and the M2 naive hybrid on both workloads across all datasets.

Facebook (FB)

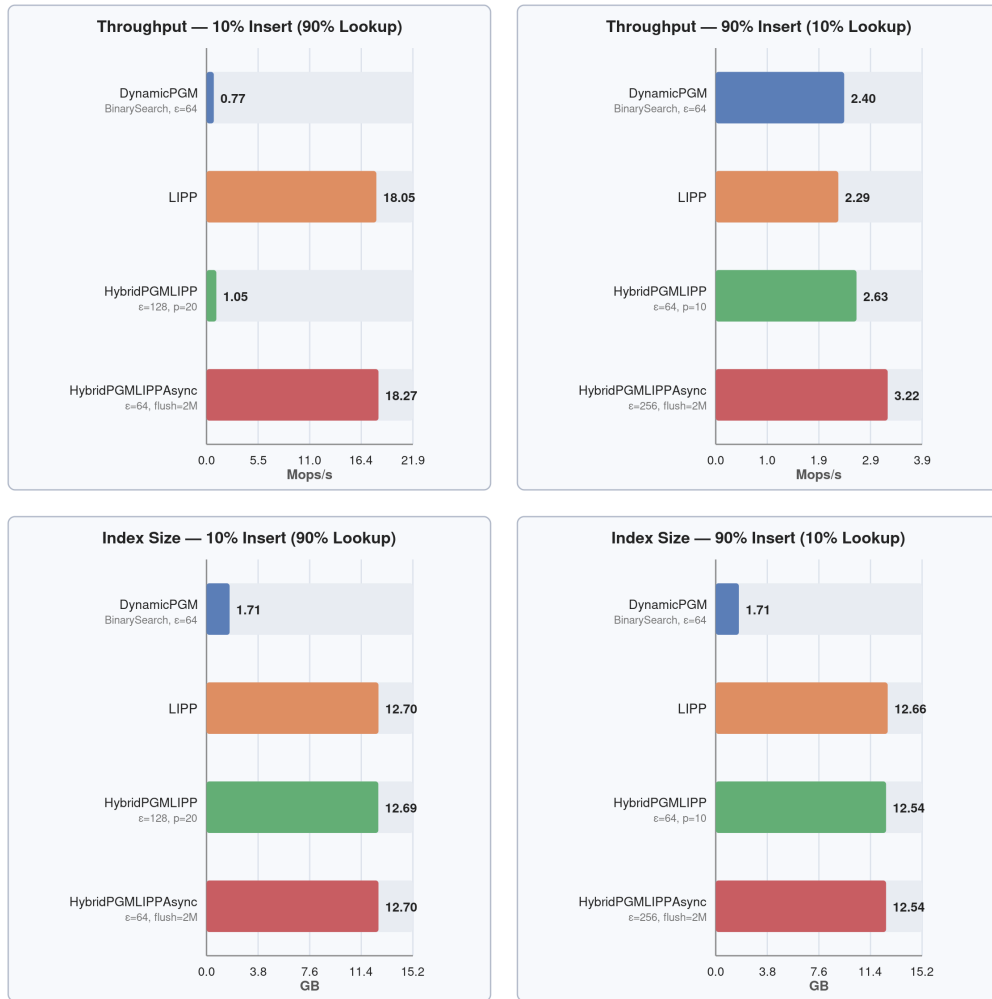


Figure 1: Facebook (FB) dataset.

Books

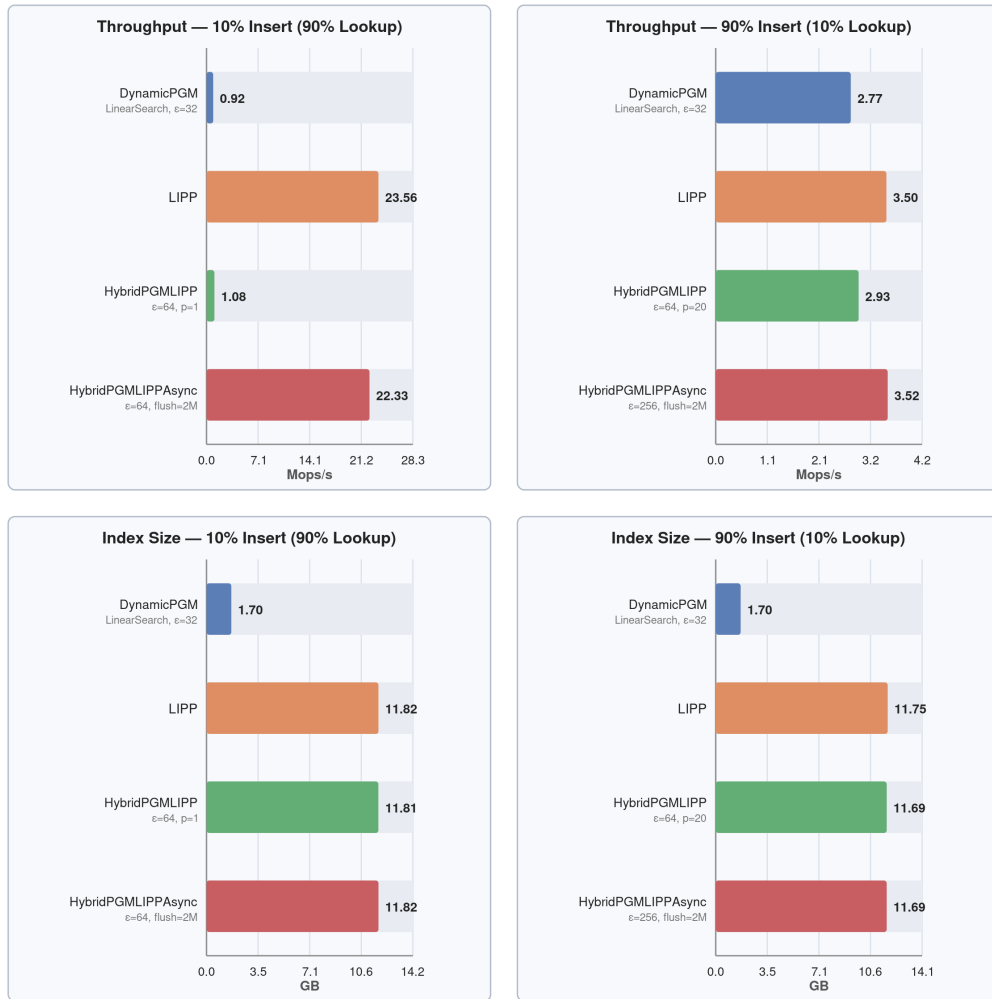


Figure 2: Books dataset.

OSMC

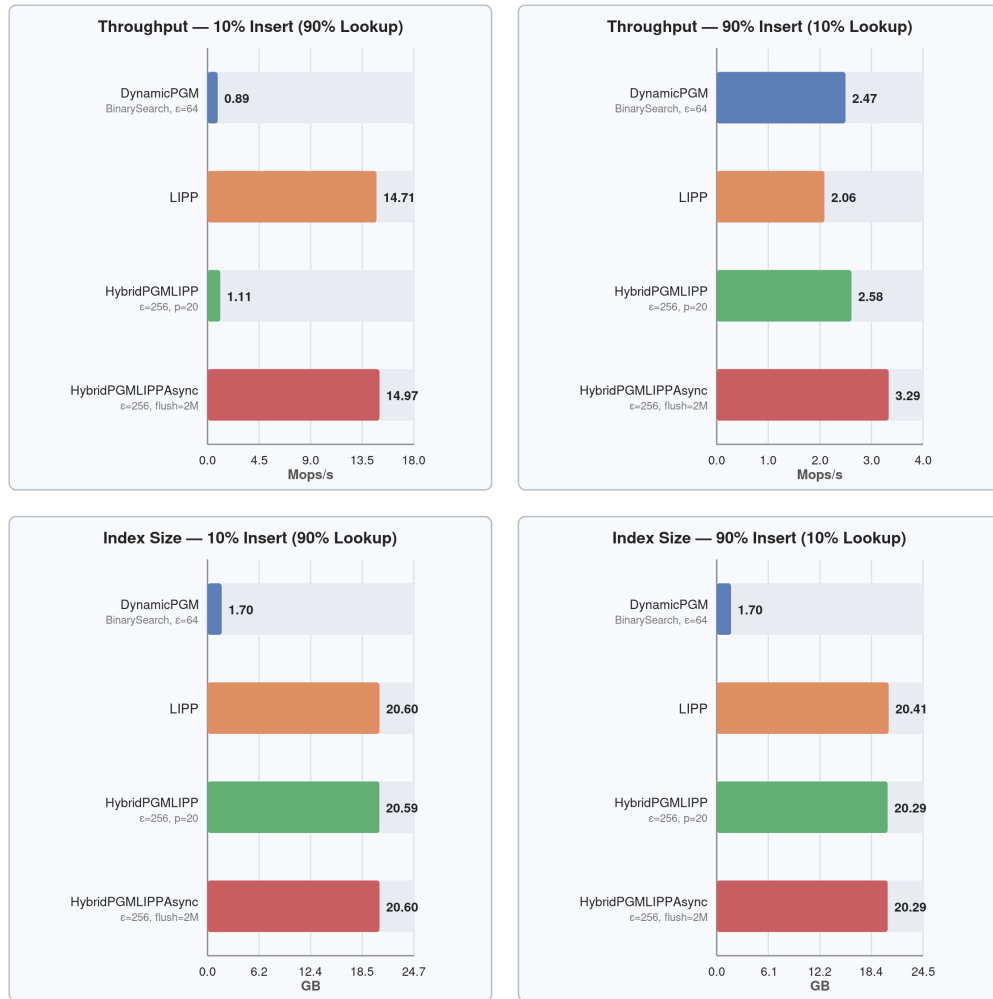


Figure 3: OSMC dataset.